Box 1. Summary of Best Practices

1. Write programs for people, not computers.
    (a) A program should not require its readers to hold more than a handful of facts in memory at once.
    (b) Make names consistent, distinctive, and meaningful.
    (c) Make code style and formatting consistent.
2. Let the computer do the work.
    (a) Make the computer repeat tasks.
    (b) Save recent commands in a file for re-use.
    (c) Use a build tool to automate workflows.
3. Make incremental changes.
    (a) Work in small steps with frequent feedback and course correction.
    (b) Use a version control system.
    (c) Put everything that has been created manually in version control.
4. Don't repeat yourself (or others).
    (a) Every piece of data must have a single authoritative representation in the system.
    (b) Modularize code rather than copying and pasting.
    (c) Re-use code instead of rewriting it.
5. Plan for mistakes.
    (a) Add assertions to programs to check their operation.
    (b) Use an off-the-shelf unit testing library.
    (c) Turn bugs into test cases.
    (d) Use a symbolic debugger.
6. Optimize software only after it works correctly.
    (a) Use a profiler to identify bottlenecks.
    (b) Write code in the highest-level language possible.
7. Document design and purpose, not mechanics.
    (a) Document interfaces and reasons, not implementations.
    (b) Refactor code in preference to explaining how it works.
    (c) Embed the documentation for a piece of software in that software.
8. Collaborate.
    (a) Use pre-merge code reviews.
    (b) Use pair programming when bringing someone new up to speed and when tackling particularly tricky problems.
    (c) Use an issue tracking tool.

Write Programs for People, Not Computers

Scientists writing software need to write code that both executes correctly and can be easily read and understood by other programmers (especially the author's future self). If software cannot be easily read and understood, it is much more difficult to know that it is actually doing what it is intended to do. To be productive, software developers must therefore take several aspects of human cognition into account: in particular, that human working memory is limited, human pattern matching abilities are finely tuned, and human attention span is short [33 37].

First, a program should not require its readers to hold more than a handful of facts in memory at once (1a). Human working memory can hold only a handful of items at a time, where each item is either a single fact or a "chunk" aggregating several facts [33,34], so programs should limit the total number of items to be remembered to accomplish a task. The primary way to accomplish this is to break programs up into easily understood functions, each of which conducts a single, easily understood, task. This serves to make each piece of the program easier to understand in the same way that breaking up a scientific paper using sections and paragraphs makes it easier to read.

Second, scientists should make names consistent, distinctive, and meaningful (1b). For example, using non-descriptive names, like a and foo, or names that are very similar, like results and results2, is likely to cause confusion.

Third, scientists should make code style and formatting consistent (1c). If different parts of a scientific paper used different formatting and capitalization, it would make that paper more difficult to read. Likewise, if different parts of a program are indented differently, or if programmers mix CamelCaseNaming and pothole_case_naming, code takes longer to read and readers make more mistakes [35,36].

Let the Computer Do the Work

Science often involves repetition of computational tasks such as processing large numbers of data files in the same way or regenerating figures each time new data are added to an existing analysis. Computers were invented to do these kinds of repetitive tasks but, even today, many scientists type the same commands in over and over again or click the same buttons repeatedly [17]. In addition to wasting time, sooner or later even the most careful researcher will lose focus while doing this and make mistakes.

Scientists should therefore make the computer repeat tasks (2a) and save recent commands in a file for re-use (2b). For example, most command-line tools have a "history" option that lets users display and re-execute recent commands, with minor edits to filenames or parameters. This is often cited as one reason command--line interfaces remain popular [38,39]: "do this again" saves time and reduces errors.

A file containing commands for an interactive system is often called a script, though there is real no difference between this and a program. When these scripts are repeatedly used in the same way, or in combination, a workflow management tool can be used. The paradigmatic example is compiling and linking programs in languages such as Fortran, C++, Java, and C# [40]. The most widely used tool for this task is probably Make (http://www..gnu.org/software/make), although many alternatives are now available [41]. All of these allow people to express dependencies between files, i.e., to say that if A or B has changed, then C needs to be updated using a specific set of commands. These tools have been successfully adopted for scientific workflows as well [42].

To avoid errors and inefficiencies from repeating commands manually, we recommend that scientists use a build tool to automate workflows (2c), e.g., specify the ways in which intermediate data files and final results depend on each other, and on the programs that create them, so that a single command will regenerate anything that needs to be regenerated.